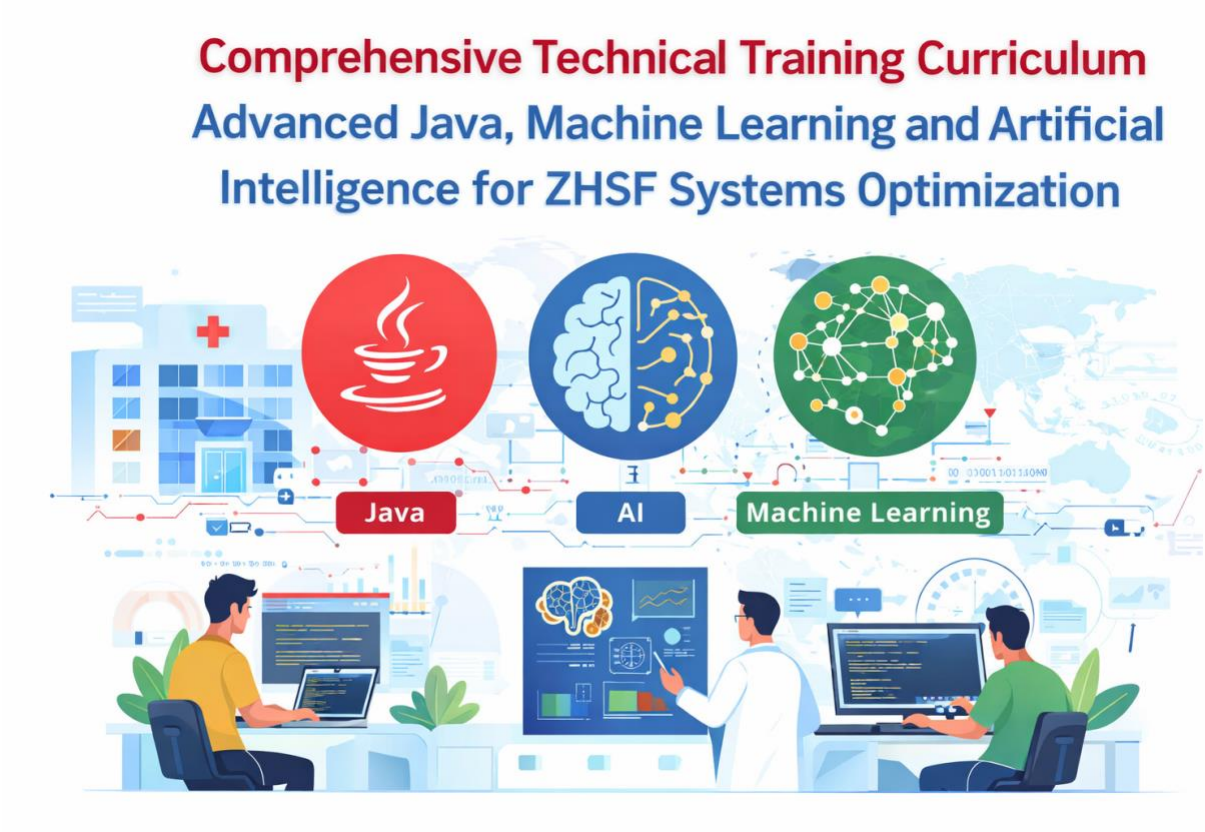

TRAINING MODULES

AI, Machine Learning & Advanced Java (Spring Boot)

Capacity Building Program for the Zanzibar Health Services Fund (ZHSF)

Prepared By	Africa Research Institute For AI (ARIFA)
Prepared For	Zanzibar Health Services Fund (ZHSF)
Date	20 March 2026
Program Duration	14 Dedicated Training Days
Total Tracks	3 Specialized Tracks
Version	Enhanced Edition v2.0

Program Overview

This document presents the enhanced and detailed training curriculum for the ARIFA-led capacity-building program commissioned by the Zanzibar Health Services Fund (ZHSF). The program is structured across three specialized tracks, each contextualized directly around ZHSF's existing technology stack, infrastructure challenges, and strategic digital health objectives.

Program Architecture

Track	Title	Focus Area	Duration	Dates
Track 1	Advanced Java & Spring Boot Optimization	Backend refactoring, API Gateway, RabbitMQ	6 Days	30 Mar – 4 Apr 2026
Track 2	Machine Learning for Health Systems	Claims anomaly detection, predictive analytics	4 Days	14–17 Apr 2026
Track 3	Artificial Intelligence & Automation	AI DevOps, log analysis, intelligent monitoring	4 Days	21–24 Apr 2026

Target Participants

- Software Developers focused on Java/Spring Boot refactoring, API design, and AI/ML integration
- Database Administrators (DBAs) focused on ML-driven query optimization and advanced PostgreSQL tuning
- System and Infrastructure Administrators focused on AI-assisted monitoring, DevOps, and HA deployments
- ICT Support Personnel focused on intelligent troubleshooting and incident response
- Business and Systems Analysts focused on leveraging ML for claims data analysis and reporting

TRACK 1: ADVANCED JAVA & SPRING BOOT OPTIMIZATION

Track Summary

Duration	6 Days (30 March 2026 – 04 April 2026)
Primary Audience	Software Developers, DBAs, Infrastructure Admins
Core Problem Addressed	CMIS 20-second load time, cron job failures, database CPU spikes, absent API Gateway
Technology Stack	Java 17+, Spring Boot 3.x, Spring Data JPA, RabbitMQ, PostgreSQL 19, Docker, Kong/Spring Cloud Gateway
Prerequisite Knowledge	Basic Java & Spring Boot familiarity, SQL fundamentals
Lab Environment	Cloned ZHSF DEV/UAT environment with anonymized production data

Track Objectives

Upon completion of Track 1, participants will be able to:

- Identify and remediate performance bottlenecks in existing Spring Boot microservices using profiling tools such as VisualVM and YourKit
- Refactor monolithic CMIS backend components into independently deployable, high-performance microservices
- Replace synchronous cron jobs with event-driven, asynchronous architectures using RabbitMQ message queues
- Design and deploy a production-grade API Gateway to manage, throttle, and secure traffic from 60+ connected hospitals via ZHIL
- Implement advanced PostgreSQL query optimization techniques including proper indexing, EXPLAIN ANALYZE usage, and connection pooling
- Configure and deploy Dockerized Spring Boot services with health checks, resource constraints, and zero-downtime rolling updates

Day-by-Day Curriculum

Day	Topic / Module	Learning Outcomes	Delivery Method
Day 1	Enterprise Java Architecture & ZHSF Profiling	Profile existing CMIS service; identify top 5 bottlenecks; understand thread dump analysis	Lecture + Hands-on Lab
Day 2	Microservices Decomposition & Refactoring	Refactor monolithic Spring Boot module into 2 microservices; apply Strangler Fig pattern	Workshop + Code Review
Day 3	Concurrency, Async Processing & RabbitMQ	Replace ZHSF cron jobs with RabbitMQ queues; implement dead-letter queues and retry logic	Guided Lab + Simulation
Day 4	API Gateway Design & Implementation	Deploy Kong/Spring Cloud Gateway; configure rate limiting, auth, and routing for ZHIL hospitals	Architecture Session + Lab
Day 5	Advanced PostgreSQL & ORM Optimization	Index foreign keys in CMIS DB; reduce full table scans; implement HikariCP connection pooling	DBA-focused Lab
Day 6	Docker, DevOps & High Availability	Configure rolling updates, health checks, Prometheus metrics; simulate hospital traffic failover	Capstone Project

Module 1.1: Enterprise Backend Architecture & CMIS Profiling

This foundational module establishes the analytical baseline for all subsequent performance work. Participants will instrument the existing CMIS Spring Boot application and generate quantitative evidence for each identified bottleneck.

Key Topics

- JVM tuning fundamentals: heap sizing, garbage collection strategies (G1GC vs ZGC) for long-running health system processes
- Thread analysis: interpreting thread dumps to identify deadlocks and contention points in multi-hospital concurrent claim submissions
- Spring Boot Actuator: enabling and securing /health, /metrics, /threaddump, and /heapdump endpoints

- Profiling with VisualVM and async-profiler: generating flamegraphs of CMIS API endpoints to isolate hot paths
- Structured logging with Logback and MDC: injecting correlation IDs to trace requests across ZHSF microservices

ZHSF-Specific Case Study: The 20-Second CMIS Load

Participants will analyze a sanitized replica of the CMIS claims query. Using EXPLAIN ANALYZE on the PostgreSQL 19 instance, the group will trace the full execution path from the Angular frontend request through the Spring Boot service layer to the database identifying N+1 query patterns, missing indexes on claims foreign keys, and synchronous nested API calls to ZHIL endpoints.

Module 1.2: Microservices Decomposition & Refactoring

Participants learn systematic approaches to breaking down monolithic Spring Boot services, with patterns directly applicable to the ZHSF claims processing pipeline.

Key Topics

- Domain-Driven Design (DDD): identifying bounded contexts within the CMIS (Claims Submission, Claims Adjudication, Hospital Onboarding, Reporting)
- Strangler Fig Pattern: incrementally migrating CMIS modules without service interruption, critical for ZHSF's 24/7 operational requirement
- Service communication patterns: synchronous REST vs asynchronous messaging, when to use each in the ZHSF context
- Spring Boot 3.x Virtual Threads (Project Loom): enabling high-throughput request handling without increasing thread pool sizes
- API versioning strategies to maintain backward compatibility with 60+ hospital ZHIL integrations

Module 1.3: Concurrency, Asynchronous Processing & RabbitMQ

This module directly addresses the cron job failures and database locks identified in the ZHSF technical assessment by replacing blocking synchronous patterns with resilient event-driven architectures.

Key Topics

- RabbitMQ topology design for ZHSF: exchanges, routing keys, and queue bindings for Claims Submission, Batch Synchronization, and Notification workflows
- Dead Letter Queues (DLQ) and retry policies: ensuring failed claim messages are never silently dropped

- Spring AMQP and @RabbitListener: replacing @Scheduled cron jobs with message-driven consumers
- Idempotency and exactly-once processing: preventing duplicate claim registrations from hospital retries
- Message persistence and durability configuration to survive RabbitMQ broker restarts

Lab 1.3: Replace ZHSF Cron Job with Async Queue

Lab #	Lab Title	ZHSF Context	Tools & Tech
L1.3	Async Claims Sync via RabbitMQ	Replace the nightly AMMIS-to-CMIS synchronization cron job with an event-driven queue	Spring AMQP, RabbitMQ 3.13, Docker Compose
L1.3b	Dead Letter & Retry Simulation	Simulate a failing hospital integration and verify DLQ capture and alert trigger	RabbitMQ Management UI, Prometheus

Module 1.4: API Gateway Design & Implementation

With 60+ hospitals connected through ZHIL, uncontrolled API traffic is a primary risk to CMIS availability. This module covers the design, deployment, and governance of a production-grade API Gateway layer.

Key Topics

- API Gateway architectural patterns: centralized entry point, cross-cutting concerns (auth, logging, rate limiting)
- Spring Cloud Gateway vs Kong: comparative analysis for ZHSF's Dockerized environment, participants will implement both
- Route configuration for ZHIL hospital endpoints: hospital-specific rate limits, timeout policies, and circuit breakers
- OAuth2 / JWT token validation at the gateway level to offload auth from individual microservices
- Canary deployments through API Gateway: routing a percentage of hospital traffic to new CMIS versions for safe rollout

Module 1.5: Advanced PostgreSQL Optimization & ORM Tuning

Database performance is the root cause of the CMIS 20-second load time. This module equips the ZHSF DBA team and backend developers with the skills to systematically optimize the PostgreSQL 19 instance.

Key Topics

- Index strategy: B-Tree, partial, and composite indexes, indexing every foreign key in the CMIS and AMMIS schemas
- EXPLAIN ANALYZE mastery: reading query execution plans, identifying Seq Scans, and eliminating them
- HikariCP connection pool tuning: calculating optimal pool sizes for ZHSF's 8-CPU backend server
- PostgreSQL partitioning: range partitioning the claims_history table by submission_date to reduce scan volumes
- Spring Data JPA N+1 prevention: using JOIN FETCH, entity graphs, and projections in claims queries
- pg_stat_statements and auto_explain: enabling continuous slow query logging for proactive DBA monitoring

Module 1.6: Docker, DevOps & High Availability

The final Track 1 module synthesizes all prior learning into a production-hardened deployment, addressing the single points of failure identified in the ZHSF architectural assessment.

Key Topics

- Docker Compose to Kubernetes migration path: evaluating readiness for ZHSF's 4-server infrastructure
- Liveness and readiness probes: preventing traffic from being routed to unhealthy CMIS service replicas
- Horizontal scaling of stateless Spring Boot services with shared PostgreSQL and RabbitMQ
- Zero-downtime rolling deployments: maintaining ZHSF service continuity during updates
- Prometheus + Grafana dashboards: pre-built panels for JVM heap, DB pool utilization, RabbitMQ queue depth, and API latency

Track 1: Lab Schedule

Lab #	Lab Title	ZHSF Context	Tools & Tech
L1.1	CMIS Performance Profiling	Profile real CMIS endpoint using VisualVM; generate flamegraph report	VisualVM, async-profiler, JDK 17
L1.2	Claims Service Microservice Extraction	Extract Claims Submission module from monolith; wire via REST + RabbitMQ	Spring Boot 3.x, IntelliJ IDEA
L1.3	RabbitMQ Async Queue Implementation	Replace nightly sync cron with event-driven queue; test with 60-hospital load	RabbitMQ, Spring AMQP, Docker
L1.4	API Gateway for ZHIL Integration	Deploy Spring Cloud Gateway with rate limiting and JWT auth for hospital routes	Spring Cloud Gateway, Keycloak
L1.5	PostgreSQL Index & Query Optimization	Add indexes to CMIS FK columns; reduce CMIS list query from 20s to <1s	PostgreSQL 19, pgAdmin, EXPLAIN
L1.6	HA Capstone: Zero-Downtime Deploy	Simulate rolling deployment under 60-hospital synthetic load; verify no dropped requests	Docker Compose, Prometheus, k6

Track 1: Assessment

Assessment Type	Description	Weight	Timing
Daily Lab Evaluation	Scored lab completion with instructor review of code and query plans	40%	End of each day
Live Optimization Challenge	Participants fix a deliberately broken CMIS configuration under time pressure	30%	Day 5
Capstone Presentation	Team presents HA deployment architecture and performance benchmark results	30%	Day 6

TRACK 2: MACHINE LEARNING FOR HEALTH SYSTEMS

Track Summary

Duration	4 Days (14 April 2026 – 17 April 2026)
Primary Audience	Business Analysts, Software Developers, DBAs
Core Problem Addressed	Manual claims review burden, undetected claim anomalies, unpredictable peak traffic from 60+ hospitals
Technology Stack	Python 3.11, scikit-learn, pandas, XGBoost, MLflow, PostgreSQL 19, FastAPI, Docker
Prerequisite Knowledge	Basic statistics, familiarity with SQL and data querying
Lab Environment	Python Jupyter environment connected to anonymized CMIS/AMMIS data warehouse

Track Objectives

Upon completion of Track 2, participants will be able to:

- Extract, clean, and engineer features from ZHSF's CMIS and AMMIS databases for ML model training
- Build, evaluate, and tune classification models to detect anomalous hospital claims with measurable precision and recall
- Develop regression and time-series models to predict peak hospital traffic loads and automate infrastructure scaling decisions
- Deploy trained ML models as REST microservices using FastAPI, integrated into the ZHSF Spring Boot architecture
- Track experiments, version models, and manage model lifecycle using MLflow on ZHSF infrastructure

Day-by-Day Curriculum

Day	Topic / Module	Learning Outcomes	Delivery Method
Day 1	Data Engineering & Feature Extraction from CMIS/AMMIS	Extract claims data via SQL; handle missing values; engineer 15+ relevant features	Lecture + Python Lab
Day 2	Claims Anomaly Detection Models	Train XGBoost classifier on claims data; tune threshold; evaluate with precision/recall/F1	ML Workshop + Lab
Day 3	Predictive Resource Scaling & Traffic Forecasting	Build Prophet/LSTM model to predict hospital submission peaks; automate alert rules	Data Science Lab
Day 4	ML Model Deployment & Integration	Containerize trained model as FastAPI service; integrate with Spring Boot CMIS endpoint	DevML Capstone

Module 2.1: Data Engineering & Feature Extraction

Effective machine learning is entirely dependent on the quality of input data. This module focuses on the specific ZHSF data ecosystem, the CMIS claims database and AMMIS administrative database, building a reliable, reproducible data pipeline for all subsequent ML work.

Key Topics

- ZHSF data landscape: schema review of CMIS claims tables, AMMIS provider tables, and ZHIL transaction logs
- SQL-to-DataFrame pipelines: efficient extraction using psycopg2 and SQLAlchemy from PostgreSQL 19
- Data quality audit: identifying null rates, duplicate claims, date inconsistencies, and out-of-range billing codes in CMIS data
- Feature engineering for claims: claim age, provider claim frequency, cost deviation from ICD-10 baselines, hospital-to-ZHSF submission lag
- Train/validation/test splits with time-based stratification to prevent data leakage in temporal health claims data
- Class imbalance handling: SMOTE oversampling and cost-sensitive learning for rare fraud/anomaly events

Module 2.2: Claims Anomaly Detection

This module directly tackles the manual claims review burden at ZHSF by building an automated anomaly detection system. Participants will train, evaluate, and interpret models capable of flagging irregular submissions from the 60+ connected hospitals.

Key Topics

- Anomaly taxonomy for health claims: duplicate claims, upcoding (billing for more expensive procedures), unbundling, phantom billing
- Supervised approach: XGBoost binary classifier trained on historically labelled ZHSF claims, feature importance analysis
- Unsupervised approach: Isolation Forest and Local Outlier Factor for detecting novel anomaly patterns without labelled data
- Model evaluation beyond accuracy: confusion matrix analysis, precision-recall tradeoff, and setting business-aligned thresholds to minimize false positives that block legitimate hospital payments
- SHAP (SHapley Additive exPlanations): providing auditors with human-interpretable explanations for why each claim was flagged
- Model fairness analysis: ensuring anomaly rates are not systematically biased against specific hospitals or ICD categories

Lab 2.2: Claims Anomaly Detection Pipeline

Lab #	Lab Title	ZHSF Context	Tools & Tech
L2.2a	Feature Engineering Notebook	Build full feature engineering pipeline on sanitized CMIS export	Python, pandas, scikit-learn
L2.2b	XGBoost Classifier Training	Train anomaly detector; achieve >85% precision on validation set	XGBoost, scikit-learn, SHAP
L2.2c	Threshold Calibration	Tune classification threshold to match ZHSF review capacity	scikit-learn, matplotlib

Module 2.3: Predictive Resource Scaling & Traffic Forecasting

ZHSF's infrastructure experiences unpredictable peaks as 60+ hospitals submit claims concurrently. This module enables infrastructure teams to anticipate demand and scale proactively rather than reactively.

Key Topics

- Time-series analysis of ZHSF API gateway logs: decomposing trend, seasonality, and noise in hospital submission patterns
- Prophet (Meta): intuitive forecasting model suitable for ZHSF's weekly/monthly hospital billing cycles
- LSTM neural networks: deep learning approach for capturing complex submission patterns from multi-hospital concurrent behavior
- Forecast confidence intervals: providing infrastructure admins with lower/upper bound resource estimates
- Integrating forecasts with infrastructure automation: triggering pre-scaling of Docker replicas or database connection pools based on predicted load
- Model monitoring: detecting forecast drift as new hospitals onboard to ZHIL and retraining triggers

Module 2.4: ML Model Deployment & ZHSF Integration

A model that exists only in a Jupyter notebook delivers zero operational value. This module bridges the gap between data science and production engineering, deploying trained models as part of the ZHSF service architecture.

Key Topics

- Model serialization: saving scikit-learn pipelines and XGBoost models with joblib and ONNX for cross-platform portability
- FastAPI REST inference service: wrapping the claims anomaly model in a lightweight, async API compatible with Spring Boot WebClient
- MLflow experiment tracking: logging all training runs, hyperparameters, and metrics; promoting best model to production registry
- Docker containerization of the inference service: integrating into ZHSF's existing Docker Compose infrastructure
- Model versioning and rollback strategy: maintaining production stability when updating models after retraining
- Data drift monitoring: detecting when the distribution of incoming claims deviates from training data, triggering retraining alerts

Track 2: Lab Schedule

Lab #	Lab Title	ZHSF Context	Tools & Tech
L2.1	CMIS Data Pipeline	Extract claims data from PostgreSQL 19; clean and engineer 15 features for ML	psycopg2, pandas, Jupyter
L2.2	Anomaly Detection Model	Train, tune, and evaluate XGBoost classifier; generate SHAP explanations	XGBoost, SHAP, scikit-learn
L2.3	Traffic Load Forecasting	Build Prophet forecasting model on ZHSF API log data; visualize 30-day forecast	Prophet, pandas, Plotly
L2.4	Model Deployment as REST API	Package anomaly model as FastAPI Docker service; test integration with CMIS	FastAPI, Docker, MLflow

Track 2: Assessment

Assessment Type	Description	Weight	Timing
Notebook Submission	Documented Jupyter notebook for data pipeline and model training with comments	35%	Day 2
Model Performance Report	Written report comparing anomaly detection model variants with business recommendations	35%	Day 3
Deployment Demo	Live demonstration of inference API responding to CMIS claim payloads	30%	Day 4

TRACK 3: ARTIFICIAL INTELLIGENCE & AUTOMATION

Track Summary

Duration	4 Days (21 April 2026 – 24 April 2026)
Primary Audience	Infrastructure Admins, System Admins, ICT Support, Developers
Core Problem Addressed	Excessive application logs causing truncation, reactive incident management, manual CI/CD processes, absence of intelligent monitoring
Technology Stack	Python, OpenTelemetry, Elastic Stack (ELK), Prometheus, Grafana, GitHub Actions, Ansible, LLM APIs
Prerequisite Knowledge	Basic Linux administration, familiarity with Docker, exposure to shell scripting
Lab Environment	ZHSF DEV environment with live application log streams and CI/CD pipeline sandbox

Track Objectives

Upon completion of Track 3, participants will be able to:

- Design and implement AI-augmented CI/CD pipelines using GitHub Actions with automated quality gates and intelligent test selection
- Deploy automated log analysis systems using Elasticsearch and LLM-based summarization to eliminate manual log triage for ZHSF infrastructure teams
- Configure intelligent observability dashboards with anomaly detection alerting in Grafana that proactively flag ZHSF system degradation before user impact
- Implement AIOps workflows that correlate events across ZHSF's application, database, and infrastructure layers to accelerate root cause analysis
- Apply AI-assisted configuration management and infrastructure-as-code to ensure consistent, auditable ZHSF environment deployments

Day-by-Day Curriculum

Day	Topic / Module	Learning Outcomes	Delivery Method
Day 1	AI-Driven DevOps & CI/CD Pipeline Intelligence	Build GitHub Actions pipeline for CMIS with AI-based test selection and security scanning	DevOps Workshop + Lab
Day 2	Automated Log Analysis & Intelligent Alerting	Deploy ELK stack; configure AI log parsing; eliminate ZHSF log truncation issue	SRE Lab + Hands-on
Day 3	Intelligent Monitoring & AIOps Observability	Build Grafana dashboards with ML-based anomaly detection for CMIS and DB metrics	Monitoring Lab
Day 4	AI-Assisted Infrastructure Automation	Implement Ansible playbooks for ZHSF server provisioning; AI-assisted config review	IaC Capstone

Module 3.1: AI-Driven DevOps & CI/CD Pipeline Intelligence

Modern CI/CD pipelines are no longer just automated build systems, they are intelligent quality gatekeepers. This module transforms ZHSF's deployment process from a manual, error-prone activity into a governed, AI-augmented pipeline.

Key Topics

- CI/CD maturity assessment: evaluating ZHSF's current deployment practices against DORA metrics (deployment frequency, lead time, MTTR, change failure rate)
- GitHub Actions pipeline architecture for ZHSF: build, test, security scan, container image build, and deploy stages for CMIS Spring Boot services
- AI-assisted test selection: using change-impact analysis to run only the test suites affected by each code change, reducing CI execution time by 60–80%
- Automated security scanning: integrating OWASP Dependency-Check and Trivy container scanning into the CMIS pipeline to prevent vulnerable dependencies from reaching production
- AI code review assistants: configuring PR review bots that flag performance anti-patterns, SQL injection risks, and missing error handling in ZHSF Java code
- Environment promotion gates: automated integration tests that must pass before any build is promoted from DEV to UAT to Production

Lab 3.1: ZHSF CI/CD Pipeline Build

Lab #	Lab Title	ZHSF Context	Tools & Tech
L3.1a	GitHub Actions Pipeline for CMIS	Build full CI/CD pipeline: compile, test, scan, Docker build, deploy to DEV	GitHub Actions, Docker, JUnit 5
L3.1b	AI Test Selection Integration	Integrate change-impact analysis tool; measure CI time reduction	pytest-testmon / Launchable

Module 3.2: Automated Log Analysis & Intelligent Alerting

The ZHSF technical assessment identified excessive application logs leading to truncation and loss of diagnostic information. This module builds a comprehensive, AI-augmented log management system that transforms logs from a liability into a diagnostic asset.

Key Topics

- Log architecture for ZHSF: structured JSON logging from Spring Boot services, PostgreSQL slow query logs, RabbitMQ message logs, and Docker container logs, all unified in a single pipeline
- Elastic Stack (ELK) deployment: Elasticsearch indexing, Logstash parsing pipelines, and Kibana dashboards for ZHSF log exploration
- Log volume management: implementing log sampling, retention policies, and index lifecycle management (ILM) to prevent storage exhaustion on ZHSF's 7TB backend server
- Pattern-based anomaly detection: Elastic ML jobs to detect statistically unusual error rates, response time spikes, and failed RabbitMQ message bursts
- LLM-powered log summarization: using Anthropic or OpenAI APIs to generate human-readable daily incident summaries from raw log streams, eliminating manual log triage for ZHSF support teams
- PagerDuty/webhook alerting integration: routing critical log anomalies to the ZHSF on-call rotation with enriched context

Module 3.3: Intelligent Monitoring & AIOps Observability

Reactive monitoring alerting after a user reports a problem is insufficient for a mission-critical health system like ZHSF. This module implements proactive, AI-enhanced observability that detects degradation before it impacts hospitals or beneficiaries.

Key Topics

- Observability pillars for ZHSF: metrics (Prometheus), logs (Elasticsearch), and traces (OpenTelemetry), the three complementary data sources
- Prometheus instrumentation: adding custom metrics to CMIS Spring Boot services, claims_submitted_total, claims_processing_duration_seconds, db_connection_pool_active
- Grafana dashboard design: pre-built ZHSF dashboards covering API gateway latency percentiles (p50/p95/p99), database CPU and connection pool saturation, RabbitMQ queue depth and consumer lag
- Grafana ML anomaly detection: configuring built-in anomaly panels to detect unusual patterns in hospital submission rates that may indicate integration failures
- Distributed tracing with OpenTelemetry: tracing a claim request from ZHIL receipt through CMIS processing to PostgreSQL commit enabling precise MTTR reduction
- SLO/SLA definition and tracking: defining ZHSF's service-level objectives (e.g., 99.5% availability, <2s claim submission response) and building error budget dashboards

Lab 3.3: ZHSF Intelligent Monitoring Stack

Lab #	Lab Title	ZHSF Context	Tools & Tech
L3.3a	Prometheus + Grafana Deployment	Deploy full observability stack; instrument CMIS with custom metrics	Prometheus, Grafana, Spring Actuator
L3.3b	Anomaly Detection Dashboard	Configure Grafana ML anomaly panels; simulate DB CPU spike and verify alert	Grafana, Prometheus, AlertManager
L3.3c	Distributed Tracing	Instrument CMIS with OpenTelemetry; trace a claim end-to-end in Jaeger UI	OpenTelemetry, Jaeger, Spring Boot

Module 3.4: AI-Assisted Infrastructure Automation

Consistent, repeatable infrastructure is the foundation of system reliability. This module equips ZHSF infrastructure administrators with AI-assisted tools to codify, audit, and automate their server provisioning and configuration management.

Key Topics

- Infrastructure as Code (IaC) principles: why ZHSF's 4-server environment requires version-controlled configuration rather than manual setup
- Ansible playbooks for ZHSF: automating the provisioning of the Database Server (128GB RAM), Backend Server (74GB RAM), Frontend Server, and Printing Server
- AI-assisted Ansible development: using LLM tools to generate, explain, and review playbooks, dramatically reducing the time to write correct YAML
- Configuration drift detection: using Ansible's, check mode and Chef InSpec to identify when server configurations have deviated from the desired state
- Secrets management: HashiCorp Vault integration for managing ZHSF database credentials, API keys, and SSL certificates, eliminating hardcoded secrets in configuration files
- Disaster Recovery automation: Ansible playbooks for ZHSF recovery runbooks, automating the restoration of services in defined RTO/RPO timeframes

Track 3: Lab Schedule

Lab #	Lab Title	ZHSF Context	Tools & Tech
L3.1	AI-Enhanced CI/CD Pipeline	Build GitHub Actions pipeline for CMIS with security scanning and AI test selection	GitHub Actions, Trivy, Docker
L3.2	ELK Log Management Stack	Deploy ELK; ingest CMIS and PostgreSQL logs; create anomaly detection job	Elasticsearch, Logstash, Kibana
L3.3	Grafana Observability Dashboard	Build ZHSF monitoring dashboards with ML anomaly detection and SLO tracking	Prometheus, Grafana, OpenTelemetry
L3.4	Ansible Infrastructure Automation	Write playbooks to provision ZHSF backend server; test DR restoration scenario	Ansible, HashiCorp Vault

Track 3: Assessment

Assessment Type	Description	Weight	Timing
Pipeline Demo	Live demonstration of CMIS CI/CD pipeline executing a code push with security scan	30%	Day 1
Log Management Setup	Operational ELK stack ingesting ZHSF log streams with anomaly job running	30%	Day 2
Capstone: Full Observability Stack	Deployed monitoring stack with Grafana dashboards, alerts, and distributed tracing	40%	Day 4

Expected Outcomes & Success Metrics

The successful delivery of all three tracks will yield the following measurable outcomes for ZHSF:

Outcome Area	Current State (Baseline)	Target State After Training
CMIS Load Performance	~20 seconds for 10 claims	<2 seconds for 100 claims
Claims Anomaly Detection	100% manual review	80%+ automated triage with AI model
Infrastructure Incidents	Reactive (post-failure)	Proactive (pre-failure AI alerts)
CI/CD Deployment	Manual & inconsistent	Automated with quality gates
Log Management	Log truncation, blind spots	Centralized ELK with AI summarization
Disaster Recovery	Undocumented manual process	Automated Ansible DR playbooks

ARIFA is fully committed to delivering this transformative capacity-building program, empowering ZHSF to build and sustain a world-class digital health infrastructure in Zanzibar.